Guía práctica · Gestión de estados en React

Asignatura: SIS0421 Servicios Web · Unidad III: Consumo de API
Caso de estudio: sincronización del sistema de Gestión Académica (React + Spring Boot)
Base: la entrega S12 (`gestion-academica` + `gestion-academica-web`). El login, el `httpClient`, el `QueryClientProvider` y las consultas de lectura ya existían; esta práctica **amplía** ese frontend sin reconstruir el login.

---

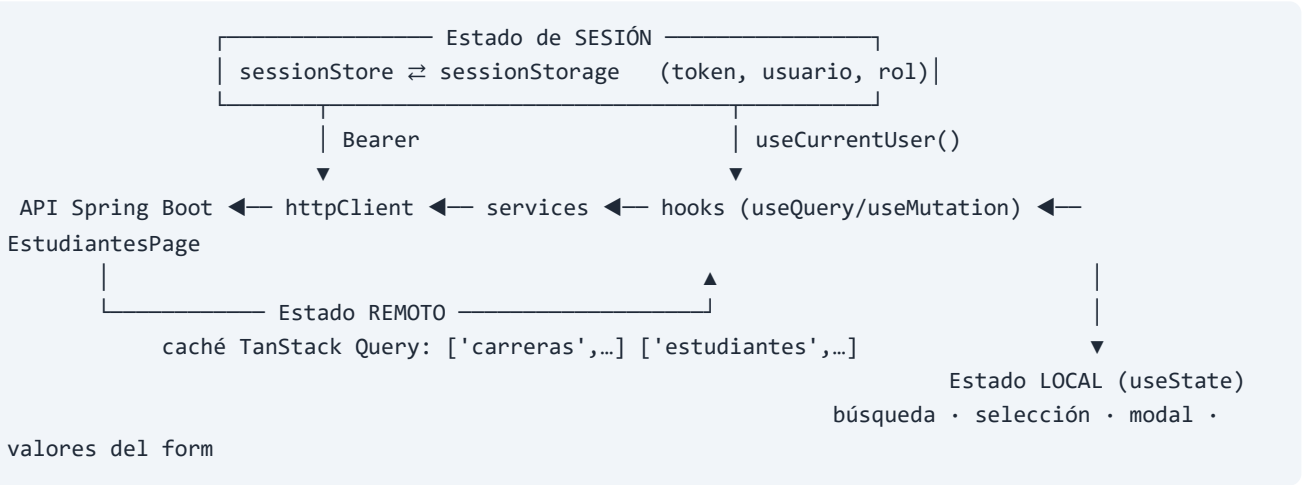
1. Parte I · Análisis de estados y diseño

Actividad 1 · Clasificación de los estados

Dato	Tipo de estado	Dónde vive	Fuente de verdad	Justificación
(a) Sesión y rol	Sesión	`shared/auth/sessionStore.ts` (store externo leído con `useSyncExternalStore`), persistido en `sessionStorage`	El JWT que entrega `POST /auth/login`	No es un dato que se pueda volver a consultar: lo produce el login una sola vez y lo leen tanto el `httpClient` (header `Bearer`) como la interfaz (`useCurrentUser`). No va en TanStack Query porque no hay `queryFn` que lo regenere.
(b) Texto de búsqueda y modal/formulario	Local de interfaz	`useState` en `EstudiantesPage` (búsqueda, fila seleccionada, formulario abierto, aviso) y en `EstudianteForm` (valores escritos y errores)	El propio componente	Solo le importan a esa pantalla, no existen en el servidor y se pierden al cerrarla sin problema.
(c) Catálogo de carreras	Remoto (servidor)	Caché de TanStack Query, clave `['carreras','list']`	`GET /api/v1/carreras`	Pertenece al servidor; se comparte entre la tabla (nombre de la carrera) y el selector del formulario. `staleTime` de 5 min porque cambia poco.

Dato	Tipo de estado	Dónde vive	Fuente de verdad	Justificación
(d) Lista de estudiantes	Remoto (servidor)	Caché de TanStack Query, claves `['estudiantes','list']` y `['estudiantes','detail',id]`	`GET /api/v1/estudiantes` (y `/{id}`)	Pertenece a la base de datos; otros usuarios pueden cambiarla. La caché se sincroniza después de cada mutación.
(e) Estado de envío del formulario	Remoto (de la mutación) + local	`useMutation` (`isPending`, `error`) + `useState` del formulario para los errores ya traducidos a cada campo	La respuesta del `POST` / `PUT`	`isPending` lo administra TanStack Query (bloquea el botón y evita envíos dobles). Cómo se pinta el error en cada campo es decisión de la interfaz, por eso se guarda localmente.

Esquema



Actividad 2 · Componentes y claves de consulta

Componente	Tipo	Responsabilidad	¿Hace peticiones?
`pages/EstudiantesPage`	Contenedor	Usa los hooks, guarda el estado local (búsqueda, selección, modal) y decide qué estado mostrar	Sí, solo a través de hooks
`components/EstudiantesTable`	Presentacional	Pinta filas; la columna **Editar** solo aparece si recibe `onEditar`	No
`components/FiltroEstudiantes`	Presentacional	Caja de búsqueda y conteo «N de M»	No
`components/EstudianteFormDialog`	Contenedor	Abre el diálogo, carga los datos existentes (`useEstudiante`) y ejecuta las mutaciones	Sí, solo a través de hooks

Componente	Tipo	Responsabilidad	¿Hace peticiones?
`components/EstudianteForm`	Presentacional	Valores, validación de cliente y errores de la API por campo	No (recibe `onGuardar`)
`carreras/components/CarreraSelect`	Presentacional	`<select>` con estados de carga / vacío / error del catálogo	No
`components/EstudianteDetalle`	Presentacional + `useEstudiante`	Panel de detalle (`GET /{id}`)	Vía hook

Claves de consulta (fábricas en `queryKeys.ts`, una por recurso):

```
carrerasKeys    = { all: ['carreras'],    list: () => ['carreras', 'list'] }
estudiantesKeys = { all: ['estudiantes'], list: () => ['estudiantes', 'list'],
                    detail: (id) => ['estudiantes', 'detail', id] }
```

Son jerárquicas: invalidar `['estudiantes']` refresca la lista **y** todos los detalles; invalidar `['estudiantes','list']` solo la lista. Carreras y estudiantes no comparten prefijo, así que una mutación de estudiantes nunca vuelve a pedir el catálogo de carreras.

Flujo: Componente → hook → servicio → `httpClient` → proxy de Vite → API. Ningún componente llama a `fetch`.

---

2. Parte II · Consultas y estados del servidor

Actividad 3. `carrerasService.listarCarreras` y `estudiantesService.listarEstudiantes` son servicios tipados (`Promise<Carrera[]>`, `Promise<Estudiante[]>`) que usan el `httpClient` existente. Así se respeta el header `Bearer`, el manejo de 401 (cierra sesión) y la traducción de errores. Los hooks `useCarreras` y `useEstudiantes` usan `useQuery` con el `signal` de cancelación.

Actividad 4. La tabla muestra nombre, correo, edad y **nombre de la carrera**, que se resuelve con el catálogo consultado (dato derivado en el render, no duplicado). Estados implementados:

Estado	Condición	Qué se ve
Carga inicial	`isPending`	Spinner «Cargando estudiantes…»
Error	`isError`	Mensaje + **Reintentar** (solo si es red/5xx); en 403 se indican los roles permitidos
Vacío	`isSuccess && data.length === 0`	«No hay estudiantes registrados» (+ botón para ADMIN)
Con datos	`isSuccess && data.length > 0`	Filtro + tabla
Filtro sin coincidencias	`filtrados.length === 0`	«Ningún estudiante coincide…» + *Limpiar búsqueda*

Filtro local por nombre o correo: `useMemo` + `Array.filter`, que devuelve un arreglo **nuevo**. Los datos en caché no se modifican y escribir en el filtro no genera peticiones.

3. Parte III · Mutaciones y sincronización

Actividad 5 · Registro (`POST /api/v1/estudiantes`). Formulario con nombre, correo, edad y un selector que usa el catálogo de carreras. Ojo con el contrato: la API **recibe** `carrera` pero **responde** `carreraId` (`EstudianteRequest` vs `EstudianteResponse`).

- **Validación de cliente** (`validation/estudianteValidation.ts`): obligatoriedad, formato de correo, edad entera positiva y carrera seleccionada. Replica las reglas del DTO para dar respuesta inmediata.
- **Validaciones de la API**, traducidas a cada campo:

Respuesta de la API	Se muestra en
400 `Validation Error` con `errors: {campo: mensaje}` (`@Valid`)	Cada campo correspondiente
400 `Business Rule Violation` «debe ser mayor de edad» (regla de dominio)	Campo **edad**
409 «El correo … ya está registrado»	Campo **correo**
404 «Carrera con id … no encontrada»	Campo **carrera**
403, falla de red, 5xx	Aviso general (con *Reintentar* si es red/5xx)

La regla «mayor de edad» se dejó **intencionalmente** solo en el backend: pertenece al dominio (`Estudiante.validateCanRegister`) y así se puede demostrar cómo se presenta un error devuelto por la API.

Actividad 6 · `useMutation` + sincronización (`hooks/useEstudiantes.ts`):

```
onSuccess: (creado) => {
  queryClient.setQueryData(estudiantesKeys.list(), (prev) => prev ? [...prev, creado] : prev); //
1. al instante
  queryClient.setQueryData(estudiantesKeys.detail(creado.id), creado);
  return queryClient.invalidateQueries({ queryKey: estudiantesKeys.list() });                  //
2. confirma con el servidor
}
```

1. Se escribe en la caché la respuesta del servidor (no los datos del formulario), así la tabla cambia de inmediato.
2. Se invalida la lista para que TanStack Query la vuelva a pedir y quede idéntica a la base de datos.

No se usa `window.location.reload()`. Al devolver la promesa de `invalidateQueries`, la mutación sigue `isPending` hasta que la lista se sincroniza. Mientras tanto el botón muestra «Guardando…», el `fieldset` queda deshabilitado y `onSubmit` ignora envíos repetidos. Al terminar aparece un aviso verde de confirmación; si falla, el error se muestra en el formulario.

Actividad 7 · Edición (`PUT /api/v1/estudiantes/{id}`). El botón **Editar** (en la fila o en el panel de detalle) abre el diálogo, que carga el registro con `GET /estudiantes/{id}` y rellena el formulario. Tras guardar, se reemplaza el registro en la lista y en su detalle y se invalida `['estudiantes']`. La tabla y el panel de detalle abierto se actualizan sin recargar. El formulario mantiene su propio estado: se monta con `key={id}` y no

se sobrescribe aunque la caché se refresque mientras se edita. Después de refrescar el navegador, los datos guardados siguen ahí (vienen de PostgreSQL) y la sesión también (`sessionStorage`).

---

4. Parte IV · Robustez y control de acceso

Actividad 8.

- **Formulario inválido** (correo incorrecto o campos vacíos): errores debajo de cada campo, foco en el primero y **los valores escritos se conservan**. Mientras se corrige, el campo se revalida al escribir.
- **Error de red:** con la API detenida, el proxy de Vite responde 5xx sin JSON y el `httpClient` lo convierte en `ApiError('network')`. El formulario muestra «No se pudo conectar con la API… Tus datos se conservaron» y el botón cambia a **Reintentar**. En la tabla y en el catálogo de carreras también aparece *Reintentar*.
- **401 vs 403**, tal como ya estaba implementado:
  - **401** (token inválido o expirado): el `httpClient` llama al handler de `App`, que cierra la sesión, limpia la caché y vuelve al login con un aviso.
  - **403** (autenticado pero sin permiso): **no** cierra la sesión. Se muestra el mensaje de permisos y no se ofrece *Reintentar*, porque el resultado sería el mismo.

Actividad 9 · Roles (`useCurrentUser()` deriva los permisos del rol de la sesión):

Rol	Lista / detalle	Nuevo / Editar	Comportamiento en la interfaz
ADMIN	✅	✅	Ve **+ Nuevo estudiante** y **Editar**
DOCENTE	✅	❌	Botones ocultos + aviso «modo de solo consulta»
ESTUDIANTE	❌ (403)	❌	Mensaje de permisos insuficientes

Estas restricciones son **solo visuales**. Se comprobó con peticiones directas que la API responde **403** a `POST` / `PUT` hechos con un token de DOCENTE, aunque se salten los botones ocultos.

---

5. Pruebas realizadas (11/11/2026)

Entorno: API Spring Boot en `:9090` + PostgreSQL 18 (servidor del proyecto en `:5433`), frontend Vite en `:5173` .

Contra la API (peticiones directas)

#	Prueba	Resultado
1	`POST /estudiantes` válido (ADMIN)	201 + `{id, …, carreraId}` ✅
2	`POST` con campos vacíos y correo inválido	400 `errors: {nombre, correo, edad, carrera}` ✅
3	`POST` con correo existente	409 «ya está registrado» ✅
4	`POST` con edad 16	400 «debe ser mayor de edad» ✅
5	`POST` con carrera 99	404 «Carrera con id 99 no encontrada» ✅

#	Prueba	Resultado
6	`PUT /estudiantes/4` (ADMIN)	200 con los datos nuevos ✅
7	`POST` y `PUT` con token de DOCENTE	403 «No tiene permisos...» ✅

Colección de Postman

`postman/GestionAcademica-S12.postman_collection.json` incorpora las peticiones **9 a 17**:

- **Operaciones de ADMIN:** POST válido, PUT, campos inválidos, correo duplicado, menor de edad y carrera inexistente.
- **Pruebas de permisos:** login de DOCENTE, POST como DOCENTE y PUT como DOCENTE (estos dos deben responder 403).

Cada petición tiene aserciones `pm.test`. El correo del alta se genera en el *pre-request* con `Date.now()`, así que la colección se puede ejecutar varias veces sin chocar con el 409. La colección completa se ejecutó con Newman: **17 peticiones y 22 aserciones, 0 fallos**.

En el navegador (prueba automatizada headless sobre la app real)

#	Escenario	Resultado
8	Login ADMIN → tabla con datos y botones Nuevo/Editar	✅
9	Filtro por correo → «1 de N»; texto sin coincidencias → estado vacío del filtro	✅
10	Enviar con correo inválido → error en el campo y el nombre escrito se conserva	✅
11	Correo duplicado → 409 mostrado en el campo correo	✅
12	Edad 16 → regla de negocio mostrada en el campo edad	✅
13	Alta válida → aviso de confirmación y fila nueva. Network: `POST /estudiantes` → `GET /estudiantes` (sin reload)	✅
14	Editar → formulario precargado con `GET /estudiantes/{id}`; guardar → `PUT` → `GET` lista + `GET` detalle	✅
15	Refrescar el navegador → el cambio persiste y la sesión sigue	✅
16	Login DOCENTE → tabla sin botones Nuevo/Editar	✅
17	Login ESTUDIANTE → 403, mensaje de permisos, la sesión **no** se cierra	✅
18	Token alterado → 401, la sesión se cierra y vuelve al login con aviso	✅

La falla de red durante el envío (Actividad 8) se demuestra manualmente: abrir el formulario, detener la API y pulsar **Registrar**.

Diagnóstico registrado (sin eludir la seguridad)

`GlobalExceptionHandler` importa `java.nio.file.AccessDeniedException` en lugar de `org.springframework.security.access.AccessDeniedException`, así que su handler de 403 nunca se ejecuta. Aun así, los 403 de `@PreAuthorize` los responde bien `RestAccessDeniedHandler`, con cuerpo JSON y `status` numérico, como se ve en las pruebas 7 y 17. No se modificó el backend en esta práctica.

---

6. Preguntas de análisis

1. ¿Por qué no conviene copiar toda la respuesta del servidor a un segundo `useState` si ya la administra TanStack Query? Porque se tendrían dos fuentes de verdad que se desincronizan. Cuando la caché se actualiza (al invalidar tras un `POST`, al reintentar o desde otra pantalla), el `useState` sigue con la copia vieja y habría que sincronizarlo a mano con `useEffect`, lo que provoca renders extra y errores sutiles. Además se pierden los estados `isPending` / `isError`, la deduplicación de peticiones y la cancelación. Lo correcto es leer `data` del hook y **derivar** lo que se necesite (filtro, nombre de la carrera) durante el render.

2. ¿Qué sucede con la vista al hacer POST si no se invalida o actualiza la consulta correspondiente? El servidor guarda el registro, pero la tabla sigue mostrando la lista anterior de la caché. El estudiante nuevo no aparece hasta que algo provoque otra consulta (pulsar *Actualizar*, volver a montar la vista o recargar la página). El usuario cree que el guardado falló y puede volver a enviarlo, lo que termina en un 409 por correo duplicado. Por eso `onSuccess` escribe la respuesta en la caché e invalida `['estudiantes','list']`.

3. ¿Cuándo debe utilizarse estado local y cuándo una query cache?

- **Query cache:** datos cuya fuente de verdad es el **servidor** (estudiantes, carreras). Pueden cambiar por acciones de otros usuarios, se comparten entre componentes y necesitan carga, error, reintento y sincronización.
- **Estado local:** datos que **solo existen en la interfaz** y le importan a un componente: texto de búsqueda, fila seleccionada, si el modal está abierto, lo que el usuario va escribiendo en el formulario.
- **Sesión:** un tercer caso. Es global pero no se consulta al servidor, por eso va en un store propio.

Regla práctica: si al recargar la página el dato debe volver a pedirse a la API, va en la query cache; si puede perderse sin problema, es estado local.

4. ¿Por qué un control de roles en React no reemplaza la autorización en Spring Security? Porque el frontend se ejecuta en el navegador del usuario, que lo controla por completo. Puede editar `sessionStorage`, modificar el JavaScript desde DevTools o llamar a la API directamente con Postman o `curl` usando su token. Ocultar un botón solo mejora la experiencia: evita mostrar acciones que van a fallar. La única barrera real es el servidor, donde `@PreAuthorize("hasRole('ADMIN')")` valida el rol dentro de un JWT **firmado** que el cliente no puede alterar sin invalidarlo. Se comprobó así: con un token de DOCENTE, `POST` / `PUT` directos responden 403.

---

7. Archivos agregados o modificados

```
gestion-academica-web/src/
├── app/App.tsx                                  (usa sessionStore en lugar de useState para la
sesión)
├── index.css                                    (estilos de filtro, diálogo, formulario y avisos)
├── shared/
│   ├── api/httpClient.ts                        (+ httpClient.put; mensaje 403 genérico)
│   └── auth/sessionStore.ts                     NUEVO · sessionStore, useSesion, useCurrentUser
└── features/
    ├── carreras/
    │   ├── queryKeys.ts                         NUEVO
    │   ├── hooks/useCarreras.ts                 (usa carrerasKeys)
    │   └── components/CarreraSelect.tsx         NUEVO
    └── estudiantes/
        ├── queryKeys.ts                         NUEVO
        ├── types.ts                             (+ EstudianteRequest)
        ├── services/estudiantesService.ts       (+ crearEstudiante, actualizarEstudiante)
        ├── hooks/useEstudiantes.ts              (+ useCrearEstudiante, useActualizarEstudiante)
        ├── validation/estudianteValidation.ts   NUEVO · validación de cliente + traducción de
errores de la API
        ├── components/EstudianteForm.tsx        NUEVO
        ├── components/EstudianteFormDialog.tsx  NUEVO
        ├── components/FiltroEstudiantes.tsx     NUEVO
        ├── components/EstudiantesTable.tsx      (+ columna Editar según rol)
        ├── components/EstudianteDetalle.tsx     (+ botón Editar según rol)
        └── pages/EstudiantesPage.tsx            (filtro, diálogo, confirmación, permisos)
```